

삼성청년 SW·AI아카데미

데이터 시각화 프로젝트

프로젝트를 위한 배경지식

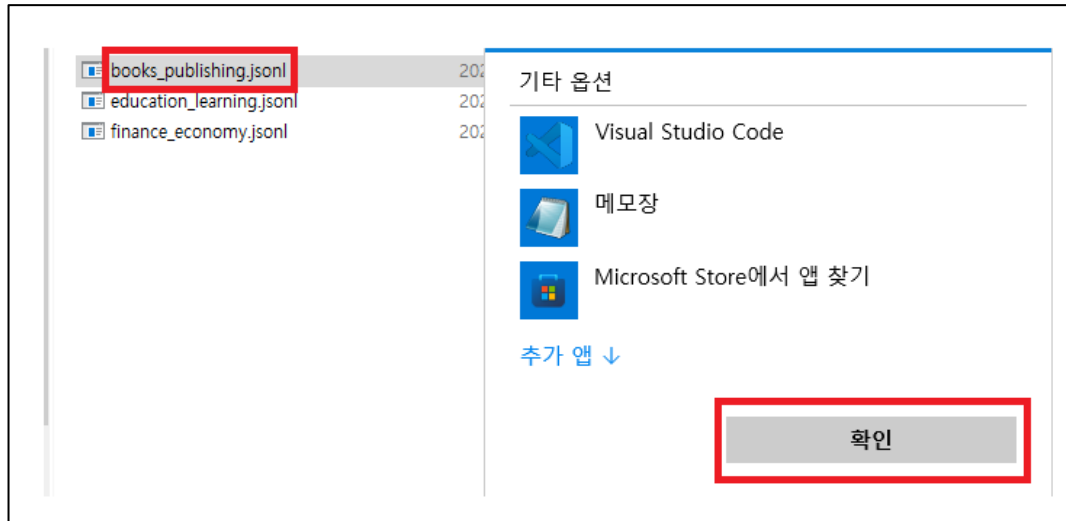
데이터 시각화(Visualization)란?

- 숫자와 텍스트로 이루어진 데이터를 그래프, 차트, 표와 같은 **시각적 요소**로 표현하는 것을 의미합니다.
- 우리는 많은 데이터를 다루지만, 사람은 숫자보다 **형태와 변화**를 더 빠르게 인식합니다.

왜 데이터 시각화가 필요할까?

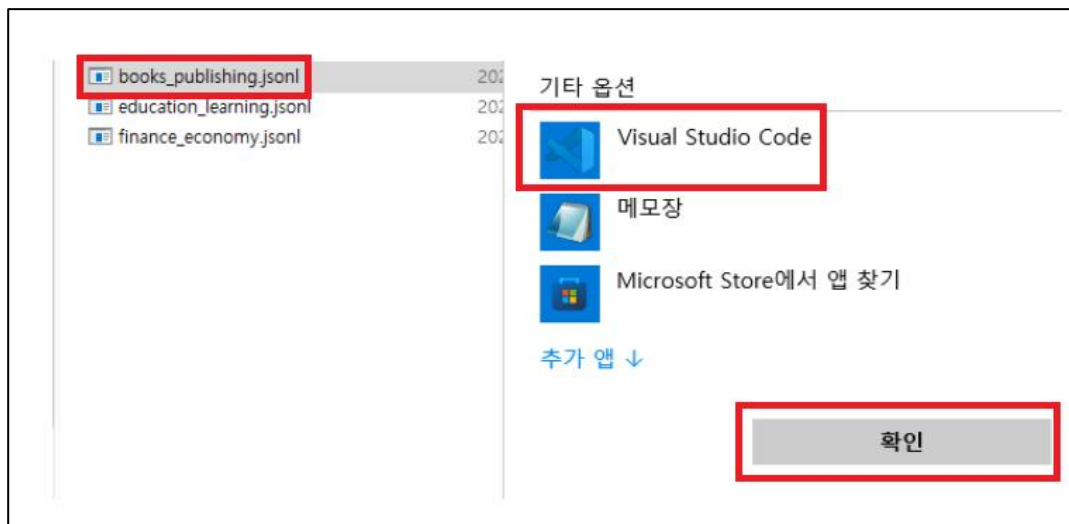
- API를 통해 데이터를 가져오는 것만으로는 사용자에게 충분한 정보를 전달할 수 없습니다.
 - JSON 데이터는 개발자에게는 익숙하지만 일반 사용자에게는 이해하기 어렵습니다
- 데이터를 표와 차트로 표현하면
 - 한눈에 흐름을 파악할 수 있고 **비교**와 **분석**이 쉬워지며 서비스의 **신뢰도**가 높아집니다

데이터 확인 하는방법 1 : 텍스트 에디터



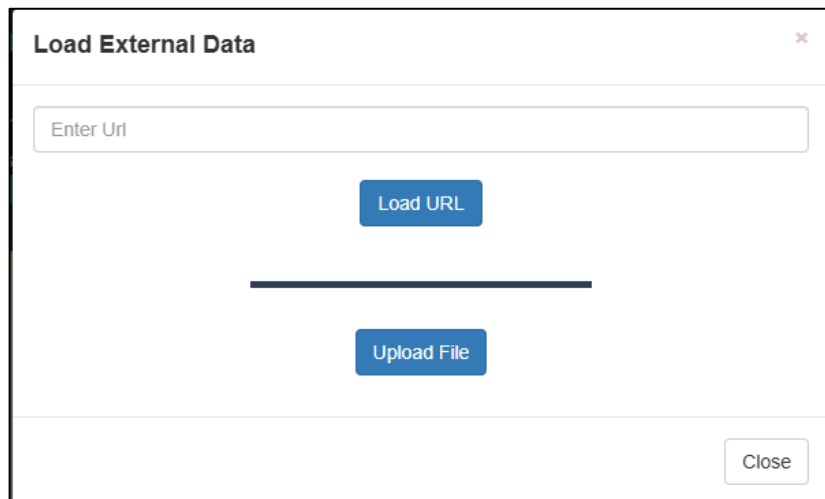
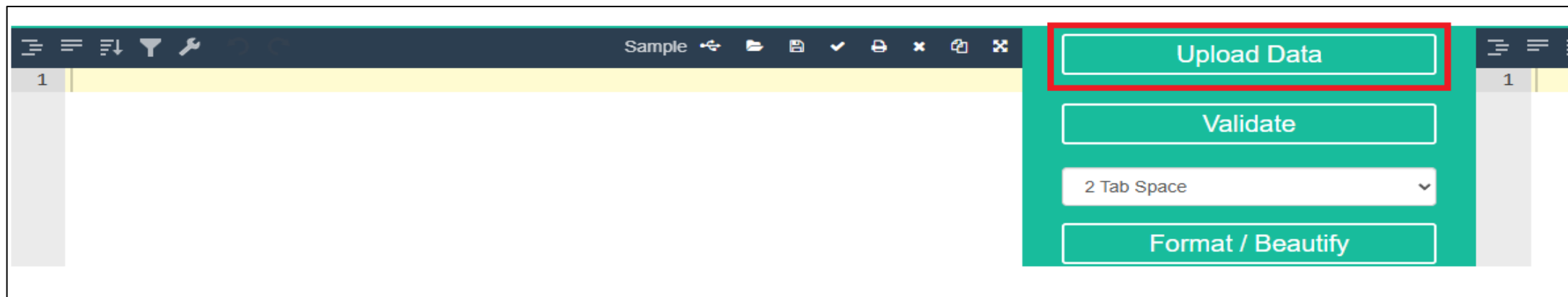
```
books_publishing.jsonl - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
{"title": "불편한 편의점", "author": "김호연", "publisher": "나무옆의자"}
{"title": "달리구트 꿈 백화점", "author": "이미예", "publisher": "팩토리"}
{"title": "하얼빈", "author": "김훈", "publisher": "문학동네", "publicati"}
{"title": "세이노의 가르침", "author": "세이노", "publisher": "데이원",
```

데이터 확인 하는방법 2 : IDE



```
() books_publishing.jsonl X
C: > Users > user > Desktop > python-track-new-pjt > 3_visualization > src > assets > data > {} books_publishing.jsonl
1 {"title": "불편한 편의점", "author": "김호연", "publisher": "나무옆의자", "publication_year": 2022}
2 {"title": "달려구트 꿈 백화점", "author": "이미예", "publisher": "팩토리나인", "publication_year": 2022}
3 {"title": "하얼빈", "author": "김훈", "publisher": "문학동네", "publication_year": 2022}
4 {"title": "세이노의 가르침", "author": "세이노", "publisher": "데이원", "publication_year": 2022}
```

데이터 확인 하는방법 3 사이트 <https://jsonformatter.org/>

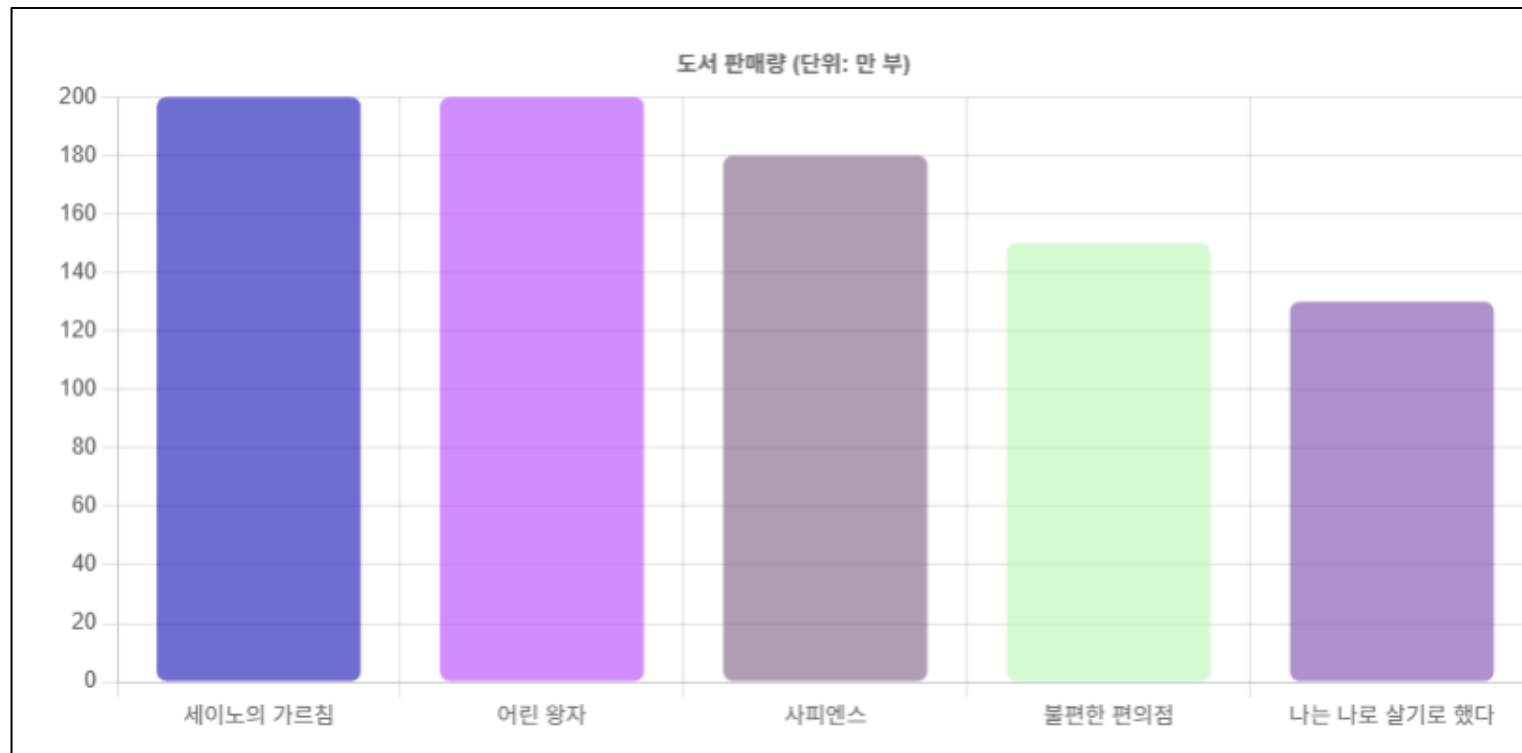


언제, 어떤 방법을 사용할까?

- 빠르게 데이터 확인 : 텍스트 에디터
- 개발 및 가공 작업 : IDE
- 구조 이해 및 시각적 확인 : 사이트
- 목적에 따라 확인 방법을 선택합니다
 - 데이터를 확인하는 방법은 여러 가지가 있지만,
 - 상황에 따라 가장 적절한 방법을 선택하는 것이 중요합니다.

데이터 양이 많아지면...

- 많은 글자를 보면서 분석하기는 힘들므로, 시각화를 하여 확인한다.



챕터의 포인트

- 핵심용어 설명
- 실습 목표
- 실습

핵심 용어 설명

핵심 용어

- **Visualization** : 데이터를 시각적인 형태로 표현하는 과정입니다.
- **Chart** : 데이터의 분포, 변화, 비교를 시각적으로 보여주는 도구입니다.
- **Dataset** : 차트에 실제로 표시되는 데이터 값들의 묶음입니다.
- **Label** : 데이터를 구분하는 기준이 되는 값입니다.(예: 연도, 출판사, 카테고리)
- **Canvas** : 차트가 그려지는 화면 영역입니다.
- **Option** : 차트의 모양, 색상, 축 등을 설정하는 정보입니다.

실습 목표

실습 목표

- 데이터를 HTML 테이블로 표현할 수 있습니다.
- Chart.js를 사용하여 기본 차트를 생성할 수 있습니다. (가볍고 러닝 커브가 낮음)
- Echarts를 활용해 시각화를 경험합니다. (기능이 압도적이고 대용량 데이터에 강함)
- 데이터를 “보여주는 서비스 화면” 을 직접 구성해봅니다.
- 단순한 데이터 출력이 아닌 데이터 기반 서비스 화면을 만드는 것이 목표입니다.

실습

Chart.js로 데이터 시각화 하기

- Chart.js는 HTML5 Canvas 기반의 차트 라이브러리입니다.
- 복잡한 설정 없이도 막대 차트, 선 차트, 원형 차트를 빠르게 만들 수 있습니다.

Chart.js로 데이터 시각화 하기

- <https://www.chartjs.org>

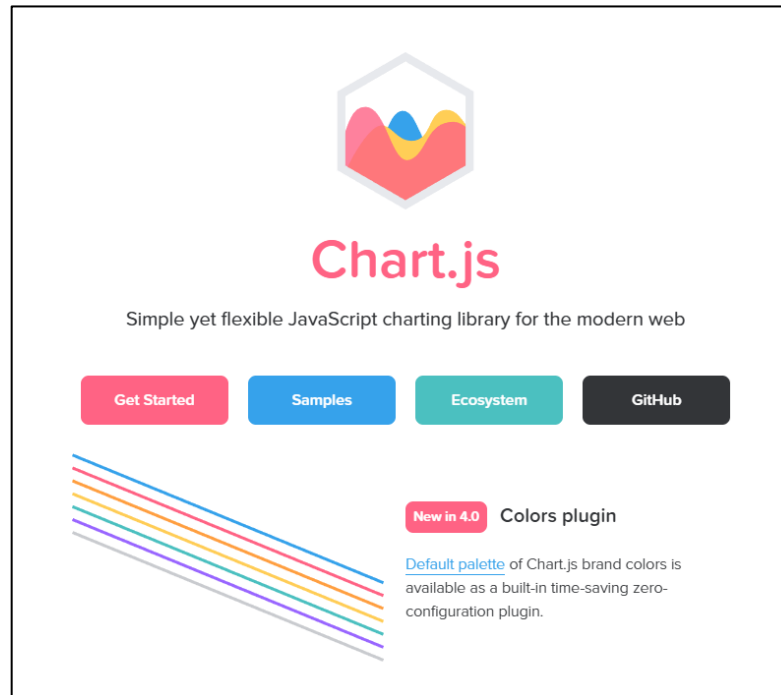


Chart.js로 데이터 시각화 하기

- 튜토리얼 코드를 적절히 활용합니다.

 Chart.js Latest (4.5.1) ▾

- Chart.js
 - Getting Started
 - Getting Started**
 - Installation
 - Integration
 - Step-by-step guide
 - Using from Node.js
 - General
 - Configuration
 - Chart Types
 - Axes
 - Developers
 - Migration

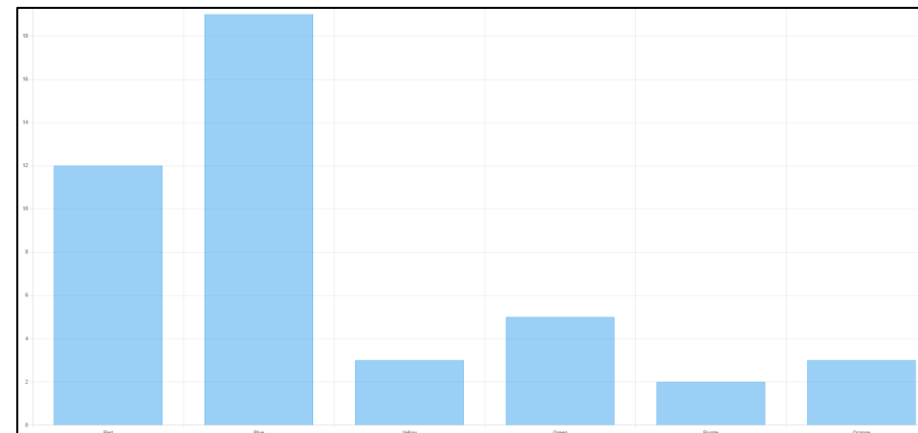
In this example, we create a bar chart for a single dataset and render it on an H

```
<div>
  <canvas id="myChart"></canvas>
</div>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script>
  const ctx = document.getElementById('myChart');

  new Chart(ctx, {
    type: 'bar',
    data: {
      labels: ['Red', 'Blue', 'Yellow', 'Green', 'Purple', 'Orange'],
      datasets: [{
        label: '# of Votes',
        data: [12, 19, 3, 5, 2, 3],
        borderWidth: 1
      }]
    },
    options: {
      scales: {
        y: {
          beginAtZero: true
        }
      }
    }
  });
</script>
```



실행 결과

Chart.js 시각화 핵심 로직

- 전체 구성을 요약하면 다음과 같습니다.
 - 차트가 그려질 영역을 <canvas> 태그를 통해 구성
 - id 를 통해 해당 요소를 가져옴
 - type, data 등을 설정하여 그래프 출력

```
<div>
  <canvas id="myChart"></canvas>
</div>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script>
  const ctx = document.getElementById('myChart');

  new Chart(ctx, {
    type: 'bar',
    data: {
      labels: ['Red', 'Blue', 'Yellow', 'Green', 'Purple', 'Orange'],
      datasets: [{
        label: '# of Votes',
        data: [12, 19, 3, 5, 2, 3],
        borderWidth: 1
      }]
    },
    options: {
      scales: {
        y: {
          beginAtZero: true
        }
      }
    }
  });
```

이 차트로 무엇을 알 수 있을까?

- 차트는 데이터를 보기 좋게 만드는 것이 목적이 아닙니다.
- 차트를 통해 우리는 다음을 확인할 수 있습니다.
 - 어떤 항목의 값이 가장 큰지
 - 데이터 간의 차이가 얼마나 나는지
 - 숫자를 직접 비교하지 않아도 되는지
- 즉, 데이터를 빠르게 이해하게 만드는 것이 목적입니다.

Vue 에 적용하기

- 프로젝트 생성

```
npm pnpm yarn bun
$ npm create vue@latest sh
```

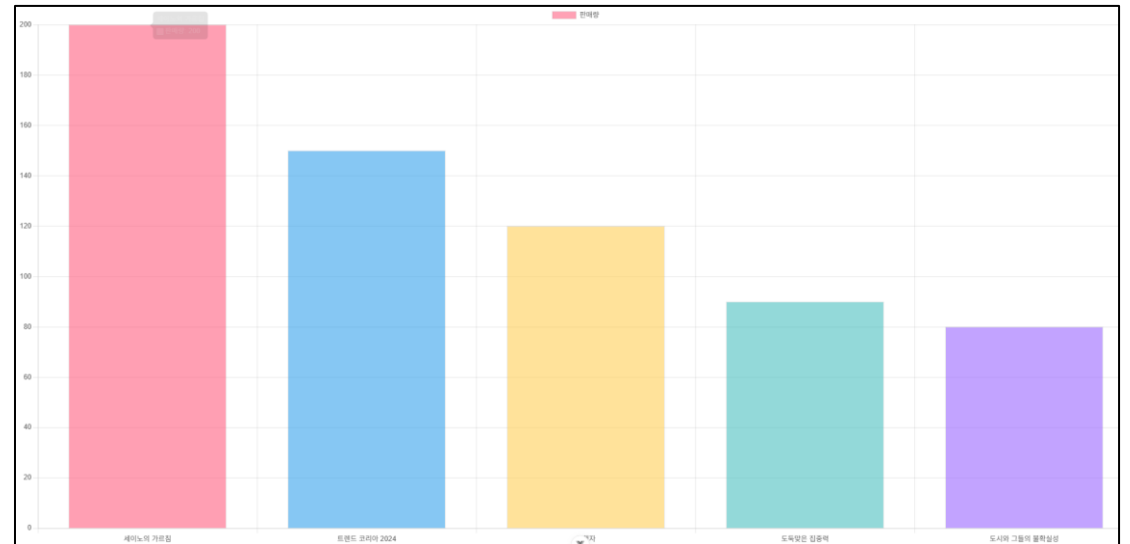
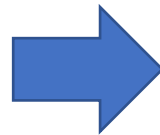
- chart.js 라이브러리 설치

```
npm install chart.js
```

Vue 에 적용하기

- 기본 코드 실행 결과 확인

```
1 <script setup>
2 import { ref, onMounted } from "vue";
3 import { Chart, registerables } from "chart.js";
4
5 // Chart.js의 모든 요소를 등록 (Bar, Line, Legend 등)
6 Chart.register(...registerables);
7
8 // 차트가 그려질 캔버스 요소 참조
9 const canvasRef = ref(null);
10
11 // 실습용 데이터 준비
12 const chartData = {
13   titleText: "도서 판매량 (단위: 만 부)",
14   labels: [
15     "세이노의 가르침",
16     "트렌드 코리아 2024",
17     "역행자",
18     "도둑맞은 집중력",
19     "도시와 그들의 불확실성"
20   ],
21   data: [200, 150, 120, 90, 80],
22   colors: [
23     "rgba(255, 99, 132, 0.6)",
24     "rgba(54, 162, 235, 0.6)",
25     "rgba(255, 206, 86, 0.6)",
26     "rgba(75, 192, 192, 0.6)",
27     "rgba(153, 102, 255, 0.6)"
28   ]
29 };
30
```



실행 결과

왜 Vue에서 onMounted를 사용할까?

- DOM이 준비되기 전엔 canvas가 없습니다.
 - 없는 곳에 그리려고 하면 버그가 발생합니다.
- mounted 시점에 차트를 생성해야 합니다.
 - “라이프사이클” 개념을 잘 활용해야 합니다.

인터랙티브(Interactive) UI 란 ?

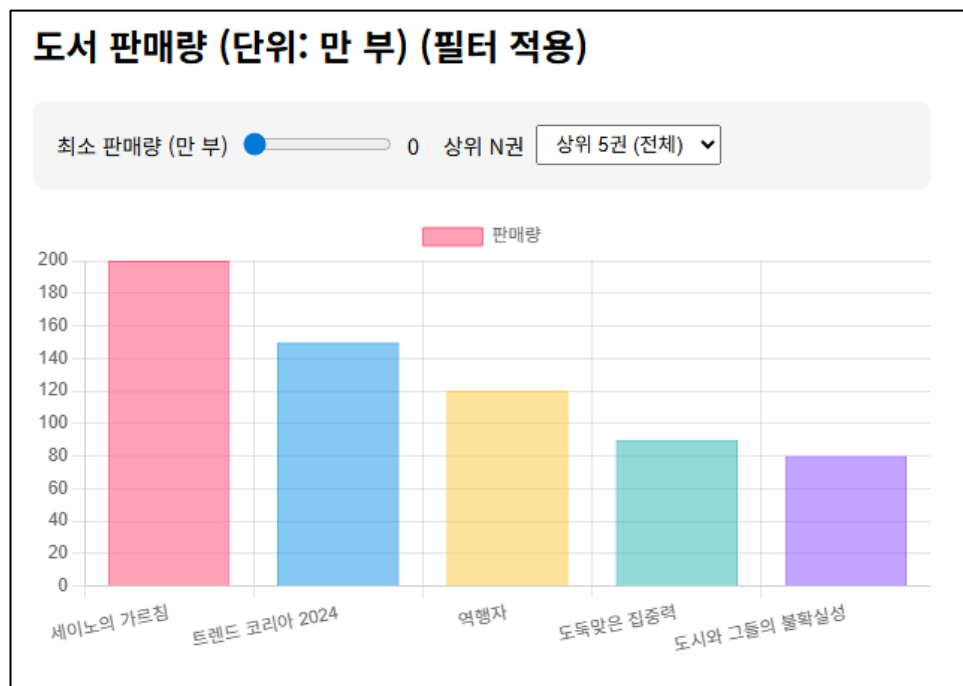
- 사용자의 입력 (마우스, 터치, 키보드 등)에 따라 화면이 실시간으로 변하는 UI
- 구성 요소 네 가지
 - 입력
 - 이벤트 처리
 - 상태
 - 렌더링

Chart.js 의 인터랙티브 수준

- 차트 라이브러리들은 대부분 기본적으로 인터랙티브 기능들을 지원함
- Chart.js
 - 툴팁 기본 지원
 - 클릭 이벤트 지원
 - 애니메이션 지원
 - 실시간 업데이트 기능

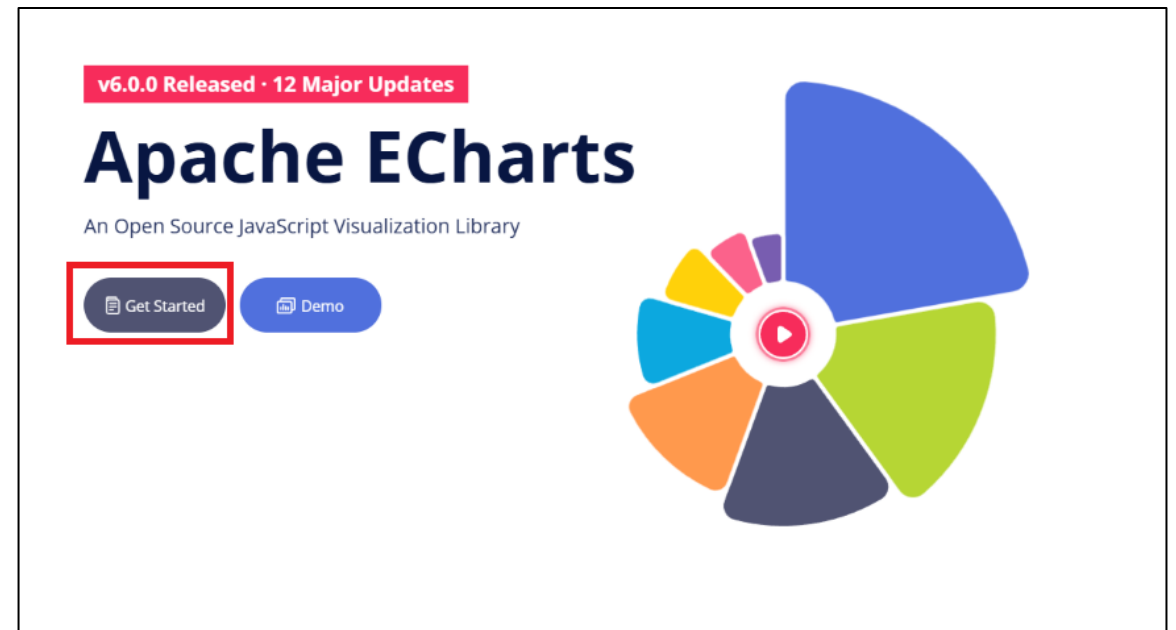
필터링 기능 추가하기

- 사용자의 입력에 따라 출력되는 차트를 실시간으로 변경할 수 있다.



Echarts로 확장된 시각화 경험하기

- 공식문서에 접속합니다.
- <https://echarts.apache.org/en/index.html>



왜 Echarts를 사용할까 ?

항목	Chart.js	ECharts
학습 난이도	낮음	중간
초기 설정	매우 간단	상대적으로 복잡
차트 종류	기본 위주	매우 다양
인터랙션	기본	매우 풍부
대규모 데이터	보통	강함
실무 대시보드	△	◎

HTML 에서 기본 차트 그리기

- CDN 방식으로 추가합니다.

Getting Apache ECharts

Apache ECharts supports several download methods, which are further explained in the next tutorial [Installation](#). Here, we take the example of getting it from the [jsDelivr](#) CDN and explain how to install it quickly.

At <https://www.jsdelivr.com/package/npm/echarts> select `dist/echarts.js`, click and save it as `echarts.js` file.

More information about these files can be found in the next tutorial [Installation](#).

INSTALL

Type:

ESM

Default

Version:

Static

```
<script src="
  https://cdn.jsdelivr.net/npm/echarts@6.0.0/dist/echarts.min.js
"></script>
```

Open in jsfiddle

Learn more

Copy URL

Copy SRI

Copy HTML

Copy HTML + SRI

HTML 에서 기본 차트 그리기

- 튜토리얼 기본 코드를 활용합니다.

Plotting a Simple Chart

Before drawing we need to prepare a DOM container for ECharts with a defined height and width. Add the following code after the `</head>` tag introduced earlier.

```
<body>
  <!-- Prepare a DOM with a defined width and height for ECharts -->
  <div id="main" style="width: 600px;height:400px;"></div>
</body>
```

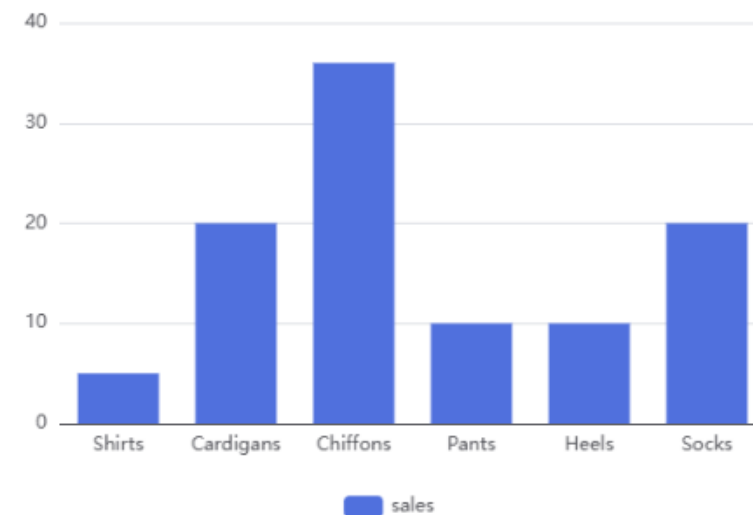
Then you can initialize an echarts instance with the `echarts.init` method and set the echarts instance with `setOption` method to generate a simple bar chart. Here is the complete code.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>ECharts</title>
    <!-- Include the ECharts file you just downloaded -->
    <script src="echarts.js"></script>
  </head>
  <body>
    <!-- Prepare a DOM with a defined width and height for ECharts -->
    <div id="main" style="width: 600px;height:400px;"></div>
    <script type="text/javascript">
      // Initialize the echarts instance based on the prepared dom
      var myChart = echarts.init(document.getElementById('main'));

      // Specify the configuration items and data for the chart
```



ECharts Getting Started Example



Echarts 시각화 핵심 로직

- 전체 구성을 요약하면 다음과 같습니다.
 - 차트가 그려질 영역을 <div> 태그를 통해 구성
 - id 를 통해 해당 요소를 가져옴
 - option, data 등을 구성 후
 - setOption 함수를 통해 설정

```
<div id="main" style="width: 600px;height:400px;"></div>
<script type="text/javascript">
  // Initialize the echarts instance based on the prepared dom
  var myChart = echarts.init(document.getElementById('main'));

  // Specify the configuration items and data for the chart
  var option = {
    title: {
      text: 'ECharts Getting Started Example'
    },
    tooltip: {},
    legend: {
      data: ['sales']
    },
    xAxis: {
      data: ['Shirts', 'Cardigans', 'Chiffons', 'Pants', 'Heels', 'Socks']
    },
    yAxis: {},
    series: [
      {
        name: 'sales',
        type: 'bar',
        data: [5, 20, 36, 10, 10, 20]
      }
    ]
  };

  // Display the chart using the configuration items and data just specified.
  myChart.setOption(option);
</script>
```

Vue 에 적용하기

- echarts 라이브러리 설치

```
npm install echarts
```

```
<script setup>
import { ref, onMounted } from "vue";
import * as echarts from "echarts";

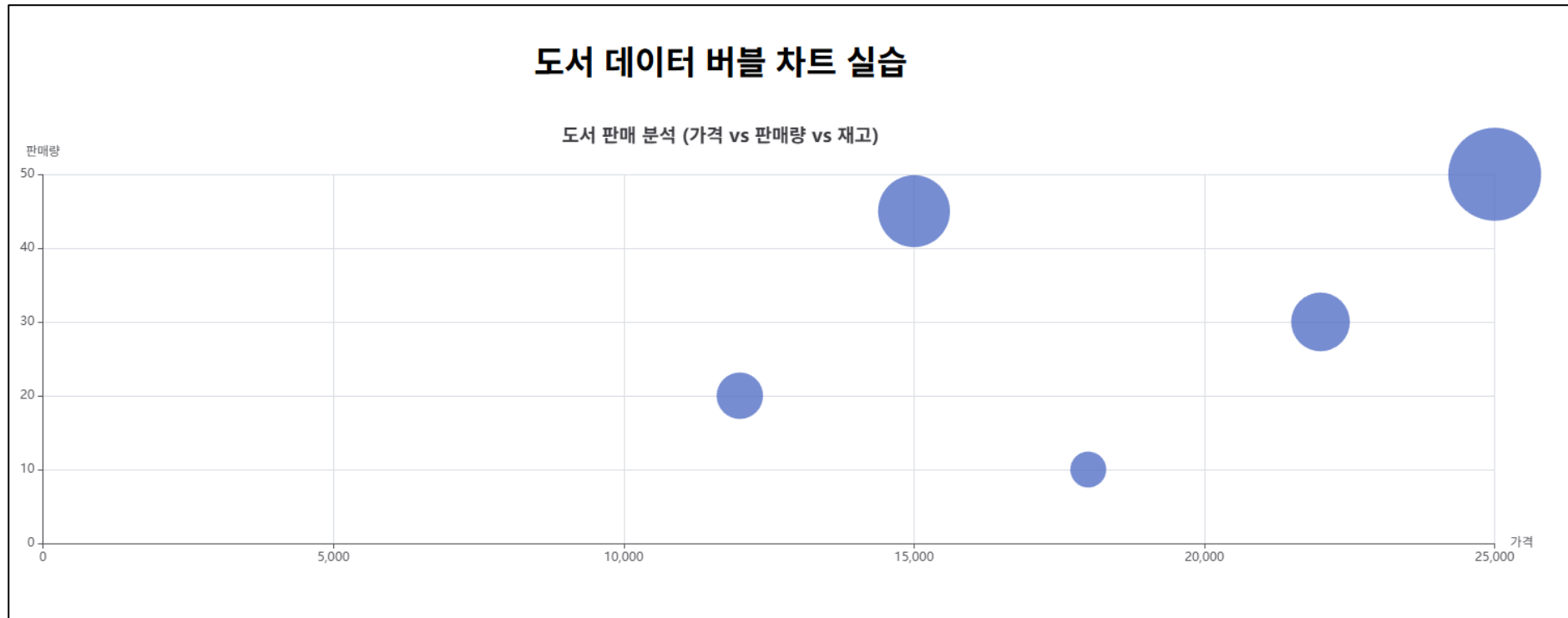
const chartRef = ref(null);

// 실습용 데이터 (X: 가격, Y: 판매량, Size: 재고)
const chartData = {
  titleText: "도서 판매 분석 (가격 vs 판매량 vs 재고)",
  // [가격, 판매량, 재고, 카테고리명]
  seriesData: [
    [12000, 20, 500, '소설'],
    [15000, 45, 1200, '자기계발'],
    [18000, 10, 300, '소설'],
    [25000, 50, 2000, 'IT/기술'],
    [22000, 30, 800, '자기계발'],
  ]
};

function renderChart() {
  if (chartRef.value) {
```

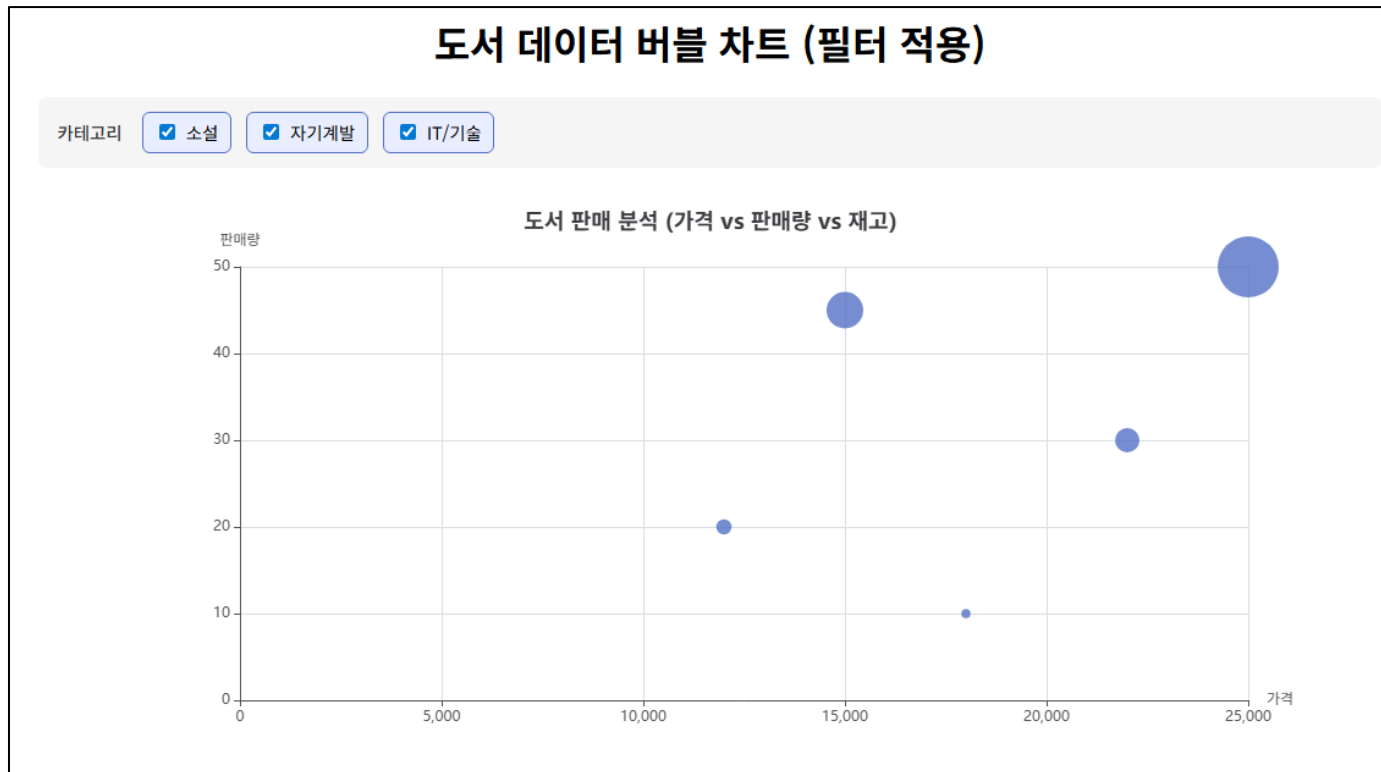

Vue 에 적용하기

- 기본 코드 실행 결과



사용자 필터링 기능 추가하기

- 카테고리 통한 필터링



왜 두 종류의 차트 라이브러리를 배웠을까?

- 상황에 따라 도구를 선택하는 것이 중요하며, 이번 프로젝트는 두가지 모두 경험해봅니다.
- 두 차트 라이브러리는 경쟁 관계가 아니라 역할이 다릅니다.
 - Chart.js - 가볍고 직관적인 통계 시각화 도구
 - 사용법이 매우 단순하고 설정도 비교적 직관적으로 가능
 - Echarts - 고급 데이터 시각화 엔진
 - 옵션 구조가 매우 방대하고 상호작용 기능이 풍부

테이블 - 또 다른 시각화 방법

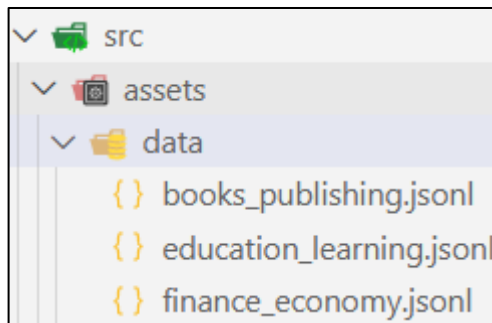
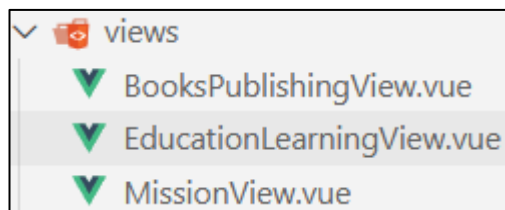
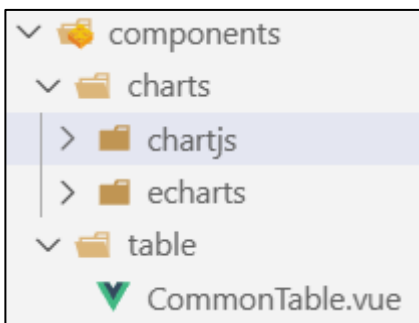
📖 베스트셀러 TOP 5				
도서명 ↕	저자 ▼	출판연도 ↕	평점 ↕	판매량 ↕
사피엔스	유발 하라리	2015년	★ 4.6	1,800,000부
어린 왕자	앙투안 드 생텍쥐페리	1993년	★ 4.9	2,000,000부
세이노의 가르침	세이노	2023년	★ 4.7	2,000,000부
불편한 편의점	김호연	2021년	★ 4.5	1,500,000부
나는 나로 살기로 했다	김수현	2016년	★ 4.3	1,300,000부

왜 테이블을 만들까?

- 차트는 보기에 화려하지만 데이터 구조를 숨기고 있습니다.
- 테이블을 통해 아래의 내용들을 미리 확인합니다.
 - 데이터의 개수
 - 항목 구성
 - 차트에 들어갈 값
- 테이블은 정확한 값을 확인하기 좋고, 차트는 비교와 흐름을 파악하기 좋습니다.
- 표와 차트를 함께 사용하는 것이 가장 좋습니다.

차트, 표가 여러 개 있을 수 있다.

- 하드 코딩한 데이터를 파일화하여 읽어옵니다.
- 공통 부분을 컴포넌트화 시킵니다.



```
import rawData from "@/assets/data/books_publishing.jsonl?raw";
import { parseJsonl } from "@/utils/parse";

const jsonlData = parseJsonl(rawData);
```

왜 컴포넌트로 분리해야 할까?

- 유지보수와 확장성
 - 차트가 여러 개면 코드가 반복됩니다.
 - 공통 부분을 컴포넌트로 만들어 중복 코드를 최소화합니다.

실습 흐름 정리

- 데이터를 확인하고 구조를 이해합니다.
- 테이블로 데이터를 정리합니다.
- Chart.js로 기본 시각화를 합니다.
- Vue에서 차트를 적용합니다.
- Echarts로 확장된 시각화를 경험합니다.
- 코드 구조를 개선하고 컴포넌트로 분리합니다.

오늘의 핵심 메시지

- 차트는 꾸미기용이 아닙니다.
 - 데이터를 이해하게 만드는 도구입니다.
- 코드보다 중요한 것은 흐름과 구조입니다.
- “어떤 파트를 쓸 지” 보다 “무엇을 보여줄 지” 먼저 고민하세요.

도전 과제 소개

필수 요구사항

- 정형 데이터를 기반으로 데이터 시각화 전체 흐름을 수행
- 데이터 구조 분석 - 가공 - 시각화 설계 - 구현 과정 수행
 - 제공된 데이터를 활용하여 주어진 요구사항을 구현
 - 이 후 자유롭게 데이터를 선택하여 진행
- 필수 기술 스택
 - Vue.js 활용

심화 과제

심화과제

- 생성형 AI 기반 시각화 분석
 - 데이터 특징 및 인사이트 도출
- UI/UX 개선
- 기능 확장